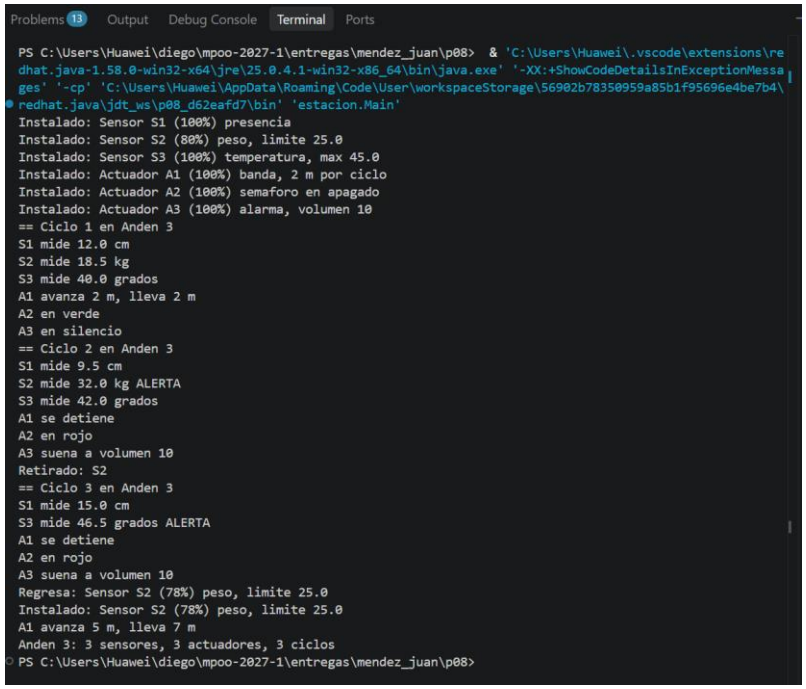
JUAN DIEGO MENDEZ REYES 426061102



1. Sobrecarga contra sobrescritura

La diferencia clave está en cómo cambia el método. La sobrecarga la usamos cuando tenemos métodos que se llaman igual pero reciben distintos datos (parámetros) dentro de la misma clase, como en BandaTransportadora con los métodos avanzar(int metros) y avanzar(), donde uno usa los metros que le pasamos y el otro toma la velocidad por defecto. En cambio, la sobrescritura (@Override) ocurre cuando una clase hija (como Alarma o Semaforo) agarra un método que heredó del padre (como reaccionar(boolean alerta)) y lo reescribe por completo para adaptarlo a lo que ella necesita hacer, manteniendo exactamente el mismo nombre y los mismos parámetros."

2. Composición contra agregación

La forma más fácil de distinguirlas es viendo quién crea a quién (new). La Bateria es composición porque vive dentro de Dispositivo: se crea con un new directo en el constructor del dispositivo, lo

que significa que si el dispositivo se destruye, la batería muere con él. Por otro lado, el sensor en la Estacion es una agregación porque los sensores se crean por fuera (en el Main) y solo se 'conectan' o instalan en la estación. Esto lo comprobamos clarísimo en el ciclo 3 con la báscula S2: cuando la sacamos de la estación para mantenimiento, el objeto no se destruye, sobrevive por su cuenta manteniendo intacta su batería al 78%, y luego puede regresar a la estación sin problema.

3. Modificadores

Al agregar la línea de prueba bascula.gastar(5); en el archivo Main.java, el compilador de Java nos marca un error rojo que dice: gastar() has protected access in Dispositivo. Esto pasa porque el método gastar está protegido protected, lo que significa que solo lo pueden usar la propia clase, sus hijos (como los sensores) o clases que estén dentro del mismo paquete. Como Main está en el paquete estación y no hereda de Dispositivo, Java le niega el acceso por seguridad para proteger los datos (y como pide la guía, esta línea ya fue borrada antes de entregar la práctica).